

[Secure E-Voting System]

End-to-end verifiable elections using blind signatures and RSA

→ CRYPTOGRAPHY PROJECT – ENSTA

HAMMOUTI Walid • FERKIOUI Akram • BOUTERBAG Amel • ZERGUINI Maylis • BELOUAHAR Sofia

[Architecture Lead]

HAMMOUTI WALID

DEF

Secure • Anonymous • Verifiable Electronic Voting

Cryptographic Implementation & Architecture

Technology Stack – Backend & Database

Backend Framework: FastAPI (Python)

- Async request handling
- Built-in OpenAPI documentation
- Type validation with Pydantic

Database: PostgreSQL + SQLAlchemy ORM

- ACID compliance
- Complex query support
- Alembic migrations

Technology Stack – Caching

Caching: Redis

- Session management
- High-performance credential lookup

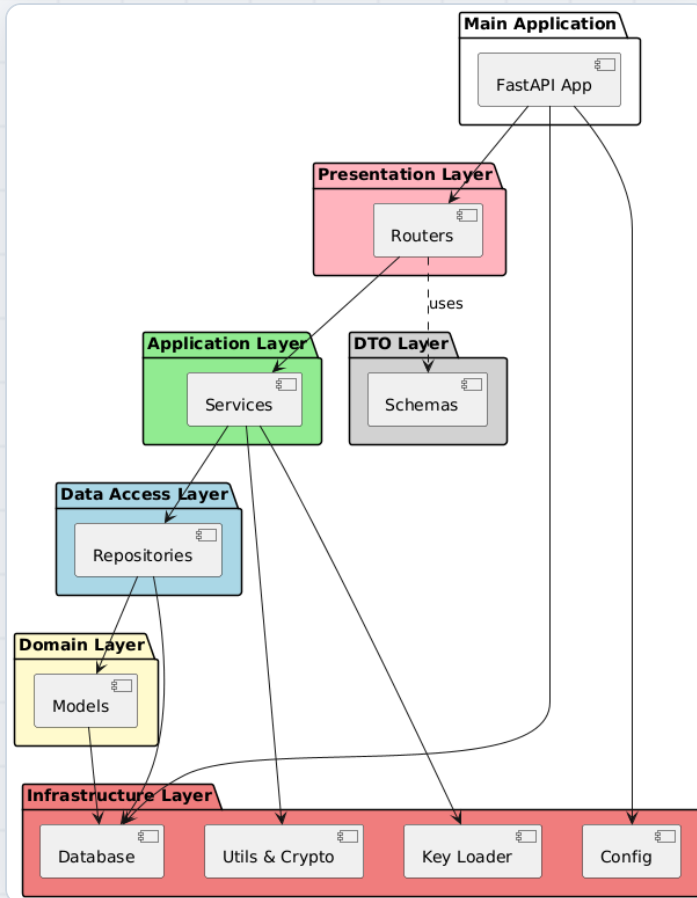
Overview

A secure, anonymous electronic voting system built with FastAPI, implementing cryptographic protocols for voter privacy and ballot integrity.

Core Principles:

- Voter anonymity through credential-based authentication
- Cryptographic vote protection using RSA encryption and blind signatures
- Transparent verification while maintaining privacy
- Automated state management and vote counting

System Architecture – Presentation & DTO Layers



Presentation Layer

- **Routers:** VotersRouter, VotingRouter, ResultsRouter, VotingSystemConfigRouter
- Entry point for all HTTP requests
- Route definition and request delegation

DTO Layer

- **Schemas:** Data Transfer Objects for validation and serialization
- VoterSchema, VoteSubmission, Ballot, N1CheckRequest/Response
- RegisterResponse, VoteSubmissionResponse, VerifyVoteRequest/Response, TallyResponse

System Architecture – Application & Data Access Layers

Application Layer

- **Services:** Core business logic and process orchestration
- VotingSystemService, AdministratorService, AnonymizerService
- CommissionerService, CounterService, EmailSenderService

Data Access Layer

- **Repositories:** Database abstraction and query interface
- VoterRepository, VotesRepository, CountedVotesRepository
- CredentialsRepository, VotingSystemConfigRepository

System Architecture – Domain & Infrastructure Layers

Domain Layer

- **Models:** ORM entities representing persistent data
- Voter, Vote, CountedVote, Credential, VotingSystemConfig

Infrastructure Layer

- **Crypto:** RSA encryption/decryption, blind signatures, digital signatures
- **Database:** SQLAlchemy engine, session factory, base model
- **KeyLoader:** Admin and counter RSA key pair management
- **Config:** Environment variables and application settings
- **Dependencies:** FastAPI dependency injection
- **Main:** Application bootstrap and middleware configuration

Database Schema – voters

1. voters



The image shows a dark-themed window titled 'cryptography' containing a database schema diagram for a table named 'voters'. The table is represented by a rectangle with a dashed border. Inside the rectangle, the name 'voters' is centered at the top. Below it, there are two rows of attributes: 'id' with the type 'INTEGER (PK)' and 'email' with the type 'STRING (UNIQUE)'. The attributes are listed on the left side of the rectangle, and their types are listed on the right side.

voters	
id	INTEGER (PK)
email	STRING (UNIQUE)

- Simple registry of eligible voters
- **No relationships** to other tables (ensures anonymity)
- Used only for credential distribution

Database Schema – credentials

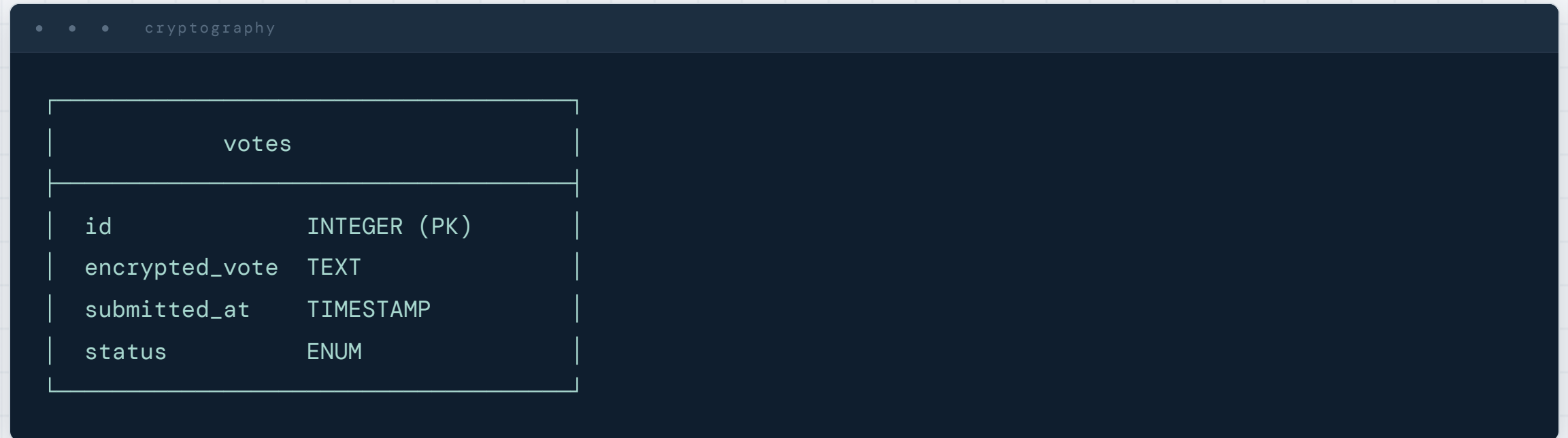
2. credentials

cryptography	
credentials	
id	INTEGER (PK)
n1	STRING (UNIQUE)
hash_n2	STRING (UNIQUE)
used	BOOLEAN

- One-time authentication tokens
- N1: plaintext for validation
- N2: hashed for receipt verification
- Uniqueness prevents collisions

Database Schema – votes

3. votes



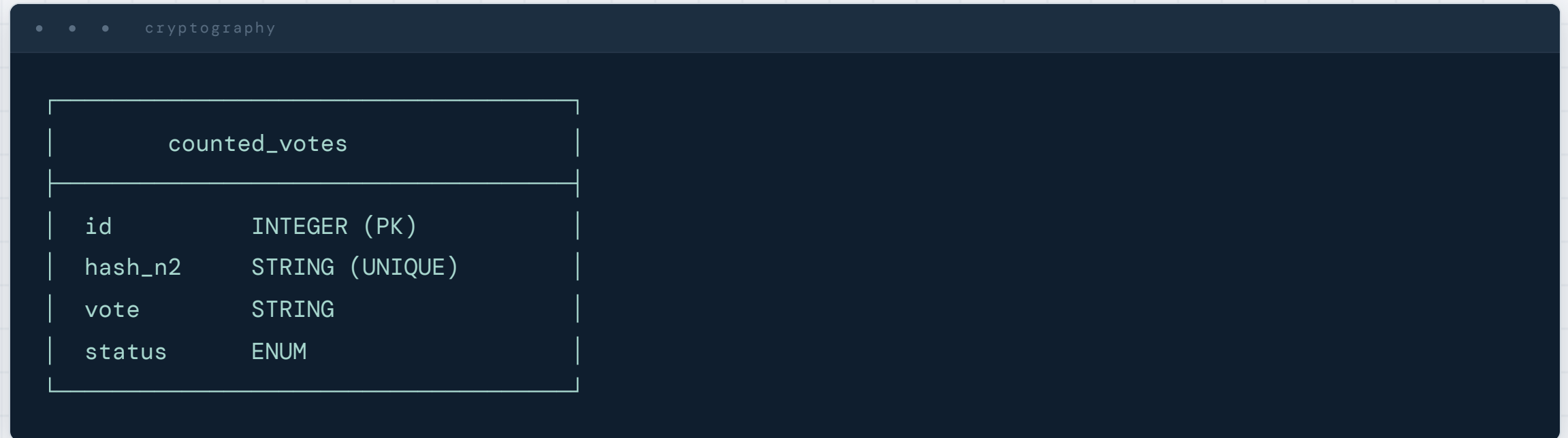
The image shows a screenshot of a database schema diagram. At the top, there are three dots followed by the text 'cryptography'. Below this, a dashed box contains the table name 'votes'. Inside this box, there is a table with four rows and two columns. The first row is the header 'votes'. The subsequent rows list the columns: 'id' (INTEGER (PK)), 'encrypted_vote' (TEXT), 'submitted_at' (TIMESTAMP), and 'status' (ENUM).

votes	
id	INTEGER (PK)
encrypted_vote	TEXT
submitted_at	TIMESTAMP
status	ENUM

- Stores encrypted ballots
- **No voter identifiers** (anonymized)
- VoteStatus: **VALID**, **REJECTED**

Database Schema – counted_votes

4. counted_votes



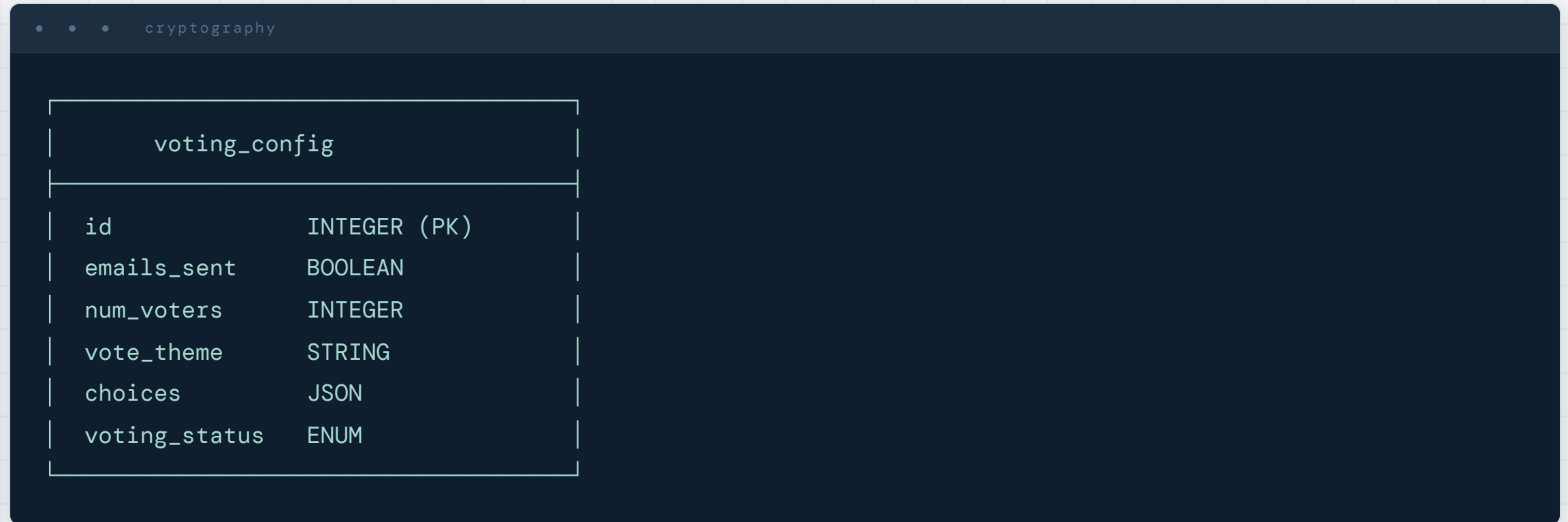
The image shows a screenshot of a database schema diagram. At the top, there are three dots followed by the text 'cryptography'. Below this, a dashed box contains the table name 'counted_votes'. Inside this box, there is a table with four rows and two columns. The first row is 'id' followed by 'INTEGER (PK)'. The second row is 'hash_n2' followed by 'STRING (UNIQUE)'. The third row is 'vote' followed by 'STRING'. The fourth row is 'status' followed by 'ENUM'.

counted_votes	
id	INTEGER (PK)
hash_n2	STRING (UNIQUE)
vote	STRING
status	ENUM

- Populated during tally phase
- CountedVoteStatus: `VALID`, `INVALID_SIGNATURE`, `INVALID_N2`
- Links receipt (N2) to verified vote

Database Schema – voting_config

5. voting_config

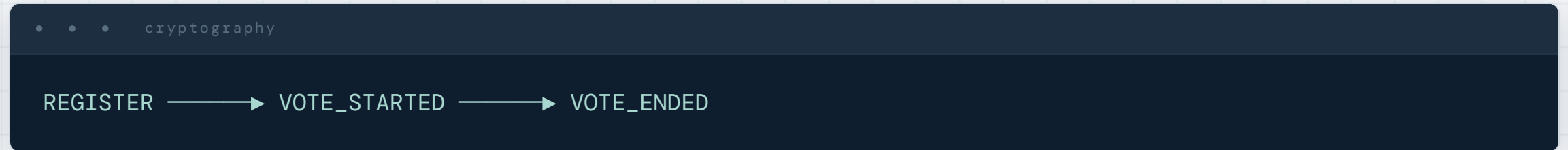


The image shows a code editor window with a dark theme. The title bar at the top has three dots and the text 'cryptography'. Inside the editor, there is a table definition for 'voting_config'. The table has six columns: 'id' (INTEGER (PK)), 'emails_sent' (BOOLEAN), 'num_voters' (INTEGER), 'vote_theme' (STRING), 'choices' (JSON), and 'voting_status' (ENUM). The table is enclosed in a light blue border.

voting_config	
id	INTEGER (PK)
emails_sent	BOOLEAN
num_voters	INTEGER
vote_theme	STRING
choices	JSON
voting_status	ENUM

- Single-row configuration table
- Controls election lifecycle
- VotingStatus: REGISTER → VOTE_STARTED → VOTE_ENDED

Election Lifecycle – State Machine



REGISTER

- Initial state after system initialization
- Admin can register voters and configure election
- Ballots not accepted

Election Lifecycle – VOTE_STARTED & VOTE_ENDED

VOTE_STARTED

- Triggered by `POST /voting/start-vote`
- Credential emails dispatched
- Voting endpoints open
- Irreversible transition

VOTE_ENDED

- Automatic: when all expected ballots received
- Manual: via `POST /voting/end-vote`
- Results become readable
- No rollback possible

Phase 1: Registration – Initial State & Flow

→ INITIAL STATE

- `voting_config.vote_status = 'registered'`
- System accepting voter registrations

→ USER REGISTRATION FLOW

1. User visits site, enters email
2. **Endpoint:** `POST /voters/register`
3. Email stored in **voters** table
4. User waits for voting to begin

Phase 1: Registration – Completion Triggers

Option A - Manual Start:

- Super-admin clicks "Start Vote"
- **Endpoint:** `POST /voting/start-vote`

Option B - Automatic Start:

- Max registrations reached (`voting_config.num_voters`)
- Automatic transition

Phase 2: Voting Preparation – Credential Distribution

→ STATUS UPDATE

– `voting_config.vote_status` → `'voting_started'`

→ CREDENTIAL DISTRIBUTION

1. **EmailSenderService** loops through **voters** table
2. Each voter receives email with:
 - **N1** (first credential - plaintext)
 - **N2** (second credential - plaintext)
3. System marks `voting_config.emails_sent = true`

Phase 2: Voting Preparation – Credential Storage

→ CREDENTIAL STORAGE

- **N1**: stored plaintext in **credentials** table
- **N2**: SHA-256 hashed, stored as `hash_n2`
- marked `used = false`

Security Note: N2 plaintext never stored in database

Phase 3: Casting Votes – Access & N1 Validation

→ STEP 1: ACCESS VOTING PAGE

1. Voter receives email with N1 and N2
2. Clicks redirect to voting page

→ STEP 2: N1 VALIDATION

1. User enters **N1**
2. **Endpoint:** `POST /voters/check_n1`
3. Validates against **credentials** table
4. If valid → Store N1 in **Redis session**

Phase 3: Casting Votes – Why Redis?

Why Redis?

- Session management between validation and submission
- Credentials not stored in browser
- Fast retrieval and automatic expiration

Phase 3: Casting Votes – Vote Submission

→ STEP 3: VOTE SUBMISSION

1. User enters:
 - N2 credential
 - Vote choice
2. Endpoint: `POST /voters/submit_vote`
3. system processes:
 - Retrieves N1 from Redis
 - Removes N1 from Redis (pop operation)
 - Validates N2 (hashed comparison)
 - Submits anonymized encrypted vote

→ ENCRYPTION PROCESS

- Ballot format: `vote||N2||random bits`

Phase 4: Vote Ending – Triggers

→ VOTING ENDS (TWO TRIGGERS)

Option A - All Votes Cast:

- Vote count equals `voting_config.num_voters`
- Automatic closure triggered

Option B - Manual End:

- Super-admin clicks "End Vote"
- **Endpoint:** `POST /voting/end-vote`

Phase 4: Vote Ending – Status & Tabulation

→ STATUS UPDATE

- `voting_config.vote_status` → `'voting_ended'`

→ RESULTS TABULATION

- **CounterService** processes all votes
- Fills **counted_votes** table
- Aggregates totals per candidate

Phase 5: Results & Verification – View Results

→ VIEW RESULTS

- **Endpoint:** `GET /results/tally`
- Displays aggregated results to all users
- Reads from **counted_votes** table
- Groups by vote choice, counts per status

Phase 5: Results & Verification – Individual Verification

→ INDIVIDUAL VOTE VERIFICATION

1. Voter wants to verify their ballot
2. **Endpoint:** `POST /results/verify-vote`
3. User provides **N2** credential
4. System hashes N2, searches **counted_votes**
5. Returns:
 - Whether matching vote found
 - Vote status: `VALID`, `INVALID_SIGNATURE`, or `INVALID_N2`
 - The vote choice (if valid)

Privacy Preserved: Only voter with N2 can verify their own vote

Security Features – Anonymity Guarantees

→ ANONYMITY GUARANTEES

- **No linkage** between `voters` and `votes` tables
- Credentials consumed after single use
- Redis session prevents browser-side credential exposure

Security Features — Audit Trail

→ AUDIT TRAIL

- All votes timestamped (`submitted_at`)
- Status tracking through lifecycle
- Invalid votes logged with failure reason
- Transparent verification post-election

Key Design Decisions — Redis & PEM Keys

→ WHY REDIS FOR N1?

- Temporary storage between validation and submission
- Automatic expiration prevents stale sessions
- Keeps credentials server-side, not in browser

→ WHY `.PEM` FILES FOR RSA KEYS?

- Standard format, works with all crypto libs
- Rotate keys by swapping a file
- OS permissions protect private keys

Key Design Decisions — Layer & Repository Splitting

→ WHY LAYER SPLITTING?

- Each layer has one reason to change
- Isolates crypto/business logic from HTTP concerns
- Makes unit testing straightforward

→ WHY A REPOSITORY LAYER FOR QUERIES?

- Keeps SQL out of business logic
- One place to change if DB is swapped
- Services call clean methods, not raw queries

Some Challenges & Solutions

→ CHALLENGE: LINKING VOTES TO VOTERS FOR VERIFICATION

Solution: N2 hash stored in `counted_votes`, voter holds plaintext

→ CHALLENGE: PREVENTING DOUBLE VOTING

Solution: N1 consumed after single use, credentials marked `used`

→ CHALLENGE: ENSURING VOTE INTEGRITY

Solution: Administrator's blind signature verified during counting

[Crypto Lead]

BOUTERBAG AMEL

DEF

Secure • Anonymous • Verifiable Electronic Voting

[Cryptographic Systems & Formulas by Role]

→ GROUP 1 | BOUTERBAG AMAL

DEF

Secure • Anonymous • Verifiable Electronic Voting

[Role 1: The Voter]

The voter uses **two cryptographic operations**:

A – Blind Signature

→ (TO GET THE BALLOT AUTHENTICATED ANONYMOUSLY)

The voter does not want the Administrator to know their vote, but still needs the Administrator's signature on it. The blind signature protocol solves this.

→ STEP 1 – CHOOSE A BLINDING FACTOR k

$$\gcd(k, N_{admin}) = 1$$

→ STEP 2 – BLIND THE BALLOT

$$m' = m \cdot k^e \pmod{N_{admin}}$$

- m = the ballot (vote + N_2 + random bits)
- e = Administrator's public key
- k = blinding factor chosen by the voter
- m' = blinded message sent to the Administrator

→ STEP 3 – UNBLIND THE SIGNATURE

After receiving m'' from the Administrator:

$$s = m'' \cdot k^{-1} \pmod{N_{admin}}$$

- k^{-1} = modular inverse of k
- s = final valid signature of the ballot

DEF

The voter now holds (m, s) – an authenticated ballot that the Administrator signed without ever seeing.

B – RSA Encryption

→ (TO SEAL THE VOTE FOR THE COUNTER)

Before sending the ballot to the Anonymizer, the voter encrypts it using the Counter's public key.

$$C = m^{e_{counter}} \pmod{N_{counter}}$$

- $e_{counter}, N_{counter}$ = Counter's public key
- C = encrypted ballot sent to the Anonymizer

DEF

To illustrate (from the exercises):

$$C = 8^3 \pmod{583} = 512$$

[Role 2: The Administrator]

The Administrator uses **RSA Digital Signature** to authenticate ballots.

A — Blind Signing

→ (DURING THE VOTING PROCESS)

The Administrator receives the blinded message m' from the voter and signs it:

$$m'' = (m')^d \pmod{N_{admin}}$$

- d = Administrator's private key
- m'' = blind signature returned to the voter

DEF

The Administrator never knows m because they only see the blinded version m' .

B — Why the Administrator cannot cheat

The blind signature formula guarantees:

$$m'' = (m \cdot k^e)^d = m^d \cdot k^{ed} \equiv m^d \cdot k \pmod{N}$$

So after unblinding:

$$s = m^d \pmod{N}$$

The Administrator signed m without knowing it — **linkage between voter and vote is impossible.**

DEF

To illustrate (from the exercises):

Blinded message received: $m' = 4$

Administrator computes: $m'' = 4^3 \pmod{55} = 9$

[Role 3: The Commissioner]

The Commissioner uses **Hash Functions** to verify N_2 codes.

A — Hash Function Formula

$$h = H(N_2)$$

The Commissioner stores only the fingerprint ***h***, never the actual ***N*₂**.

B — Key Property Used: Pre-image Resistance

Given $h = H(N_2) \Rightarrow$ impossible to compute N_2 from h

This property means:

- Even with the full list of fingerprints, the Commissioner **cannot reconstruct** any N_2
- Therefore the Commissioner **cannot create valid ballots**

C – Verification During Counting

When the Counter sends N_2 from a ballot, the Commissioner verifies:

$$H(N_2) \stackrel{?}{=} h_{stored}$$

If they match $\rightarrow$ the N_2 is valid $\rightarrow$ the vote is counted.

D – TTH (Toy Tetragraph Hash)

The transformation applied to each block of 4 characters:

$$\mathbf{v}_{new} = M \cdot (\mathbf{v}_{old} + \mathbf{b}) \pmod{26}$$

Where:

$$M = \begin{pmatrix} 1 & 1 & 1 & 1 \\ 1 & 2 & 1 & 2 \\ 1 & 1 & 2 & 2 \\ 1 & 2 & 2 & 3 \end{pmatrix}$$

DEF

To illustrate (from the exercises):

$\text{TTH}(\text{GH78IJ90KL12}) = \text{ITJU}$

[Role 4: The Anonymizer]

The Anonymizer does **not use cryptographic formulas** directly.

Its role is purely organizational:

1. Receives (N_1, C) from the voter — where C is the encrypted signed ballot
2. Verifies N_1 with the Commissioner
3. If valid $\rightarrow$ Commissioner removes N_1 from the list
4. Records the encrypted vote C

DEF

The Anonymizer never sees the vote content — it is sealed inside

$C = m^{e_{\text{counter}}} \pmod{N_{\text{counter}}}$, which only the Counter can open.

[Role 5: The Counter]

The Counter uses **two formulas** at the end of the election.

A — RSA Decryption

→ (TO OPEN THE BALLOTS)

$$m = C^{d_{counter}} \pmod{N_{counter}}$$

- $d_{counter}$ = Counter's private key
- C = encrypted ballot received from Anonymizer
- m = decrypted ballot containing the vote and N_2

B – Signature Verification

→ (TO AUTHENTICATE EACH BALLOT)

The Counter verifies the Administrator's signature s on each ballot:

$$s^{e_{admin}} \equiv m \pmod{N_{admin}}$$

- e_{admin} = Administrator's public key
- If this holds → the ballot is genuine

C – N_2 Verification

→ (WITH THE COMMISSIONER)

The Counter extracts N_2 from the ballot and sends it to the Commissioner, who checks:

$$H(N_2) \stackrel{?}{=} h_{stored}$$

Only if **both** B and C pass → the vote is counted.

DEF

To illustrate (from the exercises):

Counter verifies: $17^{27} \equiv 8 \pmod{55}$ ✓

[Complete Formula Map]

Voter	$m' = m \cdot k^e \pmod{N}$	Blind the ballot
Voter	$s = m'' \cdot k^{-1} \pmod{N}$	Unblind → get valid signature
Voter	$C = m^{e_{counter}} \pmod{N_{counter}}$	Encrypt vote for Counter
Administrator	$m'' = (m')^d \pmod{N}$	Sign blindly
Commissioner	$h = H(N_2)$	Generate fingerprint
Commissioner	$H(N_2) \stackrel{?}{=} h_{stored}$	Verify N_2 during counting
Counter	$m = C^{d_{counter}} \pmod{N_{counter}}$	Decrypt ballot
Counter	$s^{e_{admin}} \equiv m \pmod{N_{admin}}$	Verify Admin signature

DEF

"Every formula serves one purpose: making sure each role can do its job — and only its job."

[Frontend Lead]

FERKIOUI Akram

Technology Stack — Framework & Language

Next.js 16 — React / TypeScript

- App Router with file-based routing
- Full TypeScript coverage across every layer

Technology Stack — State Management

Server State: **TanStack React Query**

- Every API call wrapped in its own custom hook
- Handles loading, error, and caching automatically

Client State: **Zustand**

- Lightweight global store for vote session state
- No prop drilling, no boilerplate

Technology Stack – UI

UI: **shadcn/ui** + Tailwind CSS + Radix primitives

Custom components built on top: **stepper** **timeline** **animated-group** **text-effect** **chart** **spinner**

Dark / light mode via **next-themes**

Notifications via **Sonner**

Project Structure

• • • cryptography

```
app/           → pages (register, vote, results, admin, theme)
components/    → Admin / Vote / Results / Landing / ui
hooks/         → one hook per API operation
lib/api/       → one file per endpoint
store/         → Zustand (useVoteStore)
providers/     → React Query client
types/         → shared TypeScript types
```

22 directories · 81 files

Overview

The frontend serves two distinct audiences:

Voters — register, cast a vote, verify their ballot

System owners — manage election config, control phases, read results

The frontend is a pure data layer. All cryptographic operations happen on the backend.

Phase-Locked Routing

<code>/register</code>	Voter	<code>register</code> phase only
<code>/vote</code>	Voter	<code>vote_started</code> only
<code>/results</code>	Everyone	<code>vote_ended</code> only
<code>/results/verify-vote</code>	Voter	<code>vote_ended</code> only
<code>/admin</code>	System owner	All phases

Every page calls `useVoteStatus` on load and renders accordingly.



[Home](#)

[Vote](#)

[Results](#)

[About](#)



[Verify Results](#)

[Register](#)



Voting hasn't started yet

The voting session is not open yet. Make sure you are registered so you're ready when it begins.

[Go to Registration](#)

[Home](#)[Vote](#)[Results](#)[About](#)[Verify Results](#)[Register](#)

Voting is now open!

The voting session has started. Head over to cast your vote before the session closes.

[Go to Vote](#)

[Home](#)[Vote](#)[Results](#)[About](#)[Verify Results](#)[Register](#)

Voting is closed

The voting session has ended. You can check the final results
or confirm that your vote was counted.

[View Results](#)[Verify My Vote](#)[Back to Home](#)

Voter Flow – Phase 1: Registration

→ ROUTE: `/register`

1. Voter submits their email address
2. `POST /voters/register`
3. On success — confirmation shown, voter waits

When voting starts: The backend generates N1 and N2 and emails them to every registered voter automatically.

Voter Flow — Phase 2: Casting a Vote

→ ROUTE: `/vote` · 3-step stepper

Step 1 — Authentication Voter enters N1 → `POST /voters/check_n1` If valid: session opens, N1 stored in Zustand

Step 2 — Vote Submission Voter selects candidate + enters N2 `POST /voters/submit_vote` → sends `{ vote_choice, N2`

`}` Backend handles everything from there.

Step 3 — Confirmation Receipt screen · session saved to localStorage

Voter Flow – Phase 3: Results & Verification

→ ROUTE: `/results`

`GET /results/tally` — public results, visible to everyone Aggregated vote counts per candidate, displayed as a chart

→ ROUTE: `/results/verify-vote`

Voter enters N2 → `POST /results/verify-vote`

<code>valid</code>	Vote was counted
<code>invalid_n2</code>	N2 not found
<code>invalid_signature</code>	Ballot was tampered

System Owner Dashboard — `/admin`

Three sections:

Vote Controls Start Vote / End Vote — triggers phase transitions on the backend

Config Editor `vote_theme` · `num_voters` · `choices[ ]` Editable during registration phase only

Results Full tally after `vote_ended`

Live status badge — auto-refreshes after every action

Architecture Scalability

Feature isolation — each domain is a folder, new features don't touch existing ones

Hook-per-operation — adding an endpoint = one API file + one hook, nothing else changes

Type safety — `types/index.ts` is the single source of truth, TypeScript catches API mismatches at compile time

Phase-gating as a pattern — `useVoteStatus` is reused across all pages, any new page inherits it automatically

Key Design Decisions

- WHY PHASE-LOCKED ROUTING INSTEAD OF AUTH? No user accounts exist. Phase status is a reliable, server-sourced signal. Every page reads it fresh on load.
- WHY REACT QUERY + ZUSTAND? They don't overlap. React Query owns server data, Zustand owns local state. No data lives in both.
- WHY A STEPPER FOR VOTING? N1 then N2 — strict order. The stepper enforces the sequence and shows the voter where they are.

[Thank You]

ANY QUESTIONS?

Secure • Anonymous • Verifiable Electronic Voting